-

PART A: EXECUTIVE ARCHITECTURE SUMMARY (FINAL)

A.1 Architecture Philosophy

AfroGo Logistics will be built as a **cloud-native, microservices-based, event-driven platform** on **AWS (af-south-1, Cape Town)**, designed from day one to feel and perform like a **premium national logistics backbone**.

The core architectural goals are:

* **Elastic scale**: Seamlessly grow from hundreds to **millions of parcels per month** without redesign.

* **Cost efficiency**: Leverage **serverless and pay-per-use services** so costs closely track parcel volume.

* **High reliability and resilience**: Design for **99.9%+ uptime** on key APIs and graceful degradation under failure.

* **Real-time responsiveness**: Provide live tracking, timely notifications, and fast operations tools.

* **AI- and data-driven intelligence**: Use **ML, analytics, and automation** to optimise routing, detect fraud, improve CX, and power internal decision-making.

A.1.1 Multi-Account Cloud Strategy

AfroGo adopts a **multi-account AWS strategy** for isolation, security, and clear costing:

* **AfroGo-Dev** – development, experiments, and integration testing.

* **AfroGo-Staging** – pre-production, mirroring production as closely as possible.

* **AfroGo-Prod** – production environment serving live customers.

* (Optional later) **AfroGo-Security/Logging** – centralised security, audit, and logging services.

Benefits:

* **Hard blast-radius boundaries** (issues in dev cannot corrupt prod).

* **Clean financial visibility** per environment.

* Easier to meet **enterprise security and compliance expectations**.

A.1.2 Service Level Objectives (SLO-Driven Reliability)

Instead of generic CPU/host alarms, AfroGo's reliability is anchored on **user-centric SLOs** tied to latency and correctness of key flows.

Examples:

* **Create Parcel API SLO**

  * "**99.9% of `Create Parcel` requests complete in < 700 ms** over a rolling 30-day window."

* **Tracking API SLO**

  * "**99.9% of tracking API requests complete in < 500 ms**."

* **Error Rate SLO**

* “**< 0.5% error rate** on key transactional APIs (parcel creation, payment, driver scan).”

These SLOs:

* Drive **CloudWatch metrics and alarms** (latency percentiles and error rates, not just CPU).
* Inform autoscaling and capacity rules (provisioned concurrency, reserved capacity, etc.).
* Provide a clear, quantitative **“experience bar”** for AfroGo’s premium brand promise.

A.1.3 Event-Driven, Microservices Architecture

AfroGo’s backend is composed of **loosely coupled microservices** (Parcel, Customer, Merchant, Driver, AfroGo Point, Routing, Payments, Notifications, Tracking, Analytics, Fraud, Support, Billing, Partner Management, ML/AI) communicating via:

* **Synchronous APIs** (REST over API Gateway, occasionally WebSocket calls).
* **Asynchronous events** on **Amazon EventBridge** and **Kinesis** streams for high-volume telemetry.

This delivers:

* **High decoupling** – teams can evolve services independently.
* **Real-time reactivity** – parcel lifecycle, driver activity, and SLA breaches are propagated instantly.
* **Clear bounded contexts** – each service owns its data and logic.

A.1.4 Data Residency, Security, and Compliance

Architectural decisions are made with **POPIA** and enterprise security in mind:

* All core production data is hosted in **AWS af-south-1 (Cape Town)** by default.

* Cross-region backup and disaster recovery use **encrypted replication and snapshots** with strict access control.

* A **zero-trust service-to-service approach**:

  * Each microservice has its own IAM role with **least privilege**.

  * No broad “god-mode” roles or flat networks.

* **Encryption by default**:

  * At rest (DynamoDB, Aurora, S3, Redshift, OpenSearch).

  * In transit (TLS 1.2+ for all external communication).

The platform is designed to evolve toward **formal certifications** (e.g., ISO 27001) as the business scales.

A.1.5 Failure-Aware, Testable Architecture

AfroGo treats failure as a **normal scenario** and designs for:

* **Graceful degradation**:

  * If analytics or ML endpoints are down, core delivery flows keep working.

  * If external providers (e.g., SMS, maps) fail, there are fallback strategies.

* **Chaos and resilience testing** in non-production:

  * Periodic simulations of partial outages (e.g., slow database, failed SMS provider).

  * Verification that SLOs and alerts behave as expected.

A.1.6 Intelligence Everywhere

AfroGo’s architecture embeds **intelligence across the entire stack**:

* **Routing & optimisation**:

  * Route planning for 30+ stops per driver, capacity-aware, township-inclusive clustering.

* **Demand forecasting & SLA performance**:

  * ML-driven predictions for parcel volumes by area, peak periods, and potential SLA risks.

* **Fraud detection and risk scoring**:

  * Real-time checks for payment fraud, driver/partner anomalies, and unusual parcel patterns.

* **CX intelligence**:

  * Personalised notifications, dynamic delivery window updates, sentiment analysis on support interactions.

* **Partner performance analytics**:

  * Drivers and AfroGo Points are monitored on reliability, on-time performance, and customer ratings, powering incentives and interventions.

All of this supports AfroGo's positioning as a **premium, "intelligent" logistics fabric** for South Africa and beyond.

A.2 Technology Stack Overview (Final)

AfroGo's technology stack is optimised for **South African conditions**, **cost efficiency**, and **enterprise-grade robustness**.

A.2.1 Core Infrastructure

* **Cloud Provider**:

  * **Amazon Web Services (AWS)** – primary cloud, hosted in **af-south-1 (Cape Town)**.

* **Compute**:

  * **AWS Lambda** (primary compute for microservices – pay-per-request, auto-scaling).

  * **AWS Fargate** (for long-running or stateful processes where needed).

* **API Layer**:

  * **Amazon API Gateway (REST)** – public and internal REST APIs.

  * **Amazon API Gateway (WebSocket)** – real-time tracking and push-style server → client updates where needed.

  * **AWS AppSync (GraphQL)** – reserved for **future/optional internal dashboards** or specific complex querying use cases, not required for v1.

* **Databases & Storage**:

* **Amazon DynamoDB** – primary NoSQL store for high-throughput, low-latency transactional data (parcels, events, users, drivers, points).

* **Amazon Aurora Serverless PostgreSQL** – relational database for financial data, invoicing, and reporting that require ACID and joins.

* **Amazon ElastiCache (Redis)** – caching, driver geospatial data, sessions, rate limiting.

* **Amazon S3** – storage for documents, delivery photos, invoices, logs, data lake.

* **Content Delivery**:

* **Amazon CloudFront** – CDN for web assets, images, static content, and tracking pages.

* **Search & Analytics Engines**:

* **Amazon OpenSearch Service** – full-text search and log analytics (parcels, tickets, logs).

* **Amazon Redshift Serverless** – data warehouse for BI, executive dashboards, and advanced analytics.

* **Streaming & Eventing**:

* **Amazon EventBridge** – central event bus for business domain events.

* **Amazon Kinesis Data Streams** – high-throughput telemetry (e.g. driver GPS, barcode scans).

* **ML/AI**:

* **Amazon SageMaker** – model training and real-time inference for routing, demand forecasting, fraud, churn, and delivery-time prediction.

* **Amazon Bedrock** – generative AI capabilities (AI support assistant, summarisation, insights) as needed.

A.2.2 Frontend Technologies

* **Mobile Apps (Customer, Driver, AfroGo Point)**:

  * **React Native (TypeScript)** – single codebase for Android and iOS.

  * Integrated with:

    * **Firebase Cloud Messaging (FCM)** for push notifications (free).

    * Native deep-links to **Google Maps / Waze** for turn-by-turn navigation.

* **Web Apps**:

  * **Next.js (React, TypeScript)** – for:

    * Public marketing site (pricing, coverage, onboarding forms).

    * Web-based customer and merchant portals.

    * Internal operations/admin dashboards.

  * **Tailwind CSS** and a shared design system for a consistent, premium look and feel.

* **Internationalisation (i18n)**:

  * Built-in language support using a **key-based translation system** with JSON translation files.

  * **Languages**: English (default), Zulu, Xhosa, Sotho, Afrikaans (expandable).

  * Users can select language in the app/portal, and the UI switches accordingly.

A.2.3 Backend Application Stack

* **Languages & Frameworks**:

  * **Node.js (TypeScript)** – primary language for microservices.

  * **NestJS + Fastify** – standardised HTTP framework across all microservices, providing:

    * Shared logging, metrics, error handling, and auth middleware via an internal `@afrogo/core` library.

  * **Python** – for data processing and ML jobs (SageMaker, ETL, analytics scripts).

* **Patterns**:

  * RESTful service design with clear, versioned endpoints.

  * Event-driven patterns for asynchronous workflows (parcel lifecycle, notifications, SLA checks).

  * Strict boundary contexts per microservice (Customer, Parcel, Driver, etc.).

A.2.4 DevOps, CI/CD, and Infrastructure as Code

* **Infrastructure as Code (IaC)**:

  * **AWS CDK (TypeScript)** – primary IaC layer for all stacks:

    * Network, security groups, VPC.

    * Databases (DynamoDB, Aurora, Redis).

    * API Gateway, Lambda, EventBridge, Kinesis.

    * Analytics & ML infrastructure (Redshift, S3 data lake, SageMaker).

* **CI/CD**:

* **GitHub Actions** – main CI/CD engine:

  * Linting, unit tests, integration tests.

  * Build and deploy of microservices to Lambda/Fargate.

  * Build and deploy of web frontends to S3 + CloudFront.

  * Versioned, controlled deployments to dev, staging, and prod accounts.

* **Deployment Strategy**:

  * **Blue/green / canary deployments** for Lambda via CodeDeploy or equivalent patterns for safe releases.

  * Feature flags (via config / AppConfig / simple flag service) to gradually enable features without redeploy.

* **Environments**:

  * **Dev** – rapid iteration.

  * **Staging** – pre-production, close to prod configuration.

  * **Prod** – locked-down, limited access, tied to live SLOs.

A.2.5 Observability and Logging

* **Monitoring & Metrics**:

  * **Amazon CloudWatch** – core metrics and dashboards for:

    * API latency and error rates (per endpoint).

    * Lambda invocations, errors, cold starts.

  * DynamoDB throughput and throttling.

  * Custom business metrics (parcels created, delivered, SLA breaches).

* **Amazon X-Ray** – distributed tracing for end-to-end request flows.

* **Logging**:

  * All microservices emit **structured JSON logs** with correlation IDs.

  * Logs shipped from CloudWatch Logs to **OpenSearch** for search and analysis (via Firehose/Lambda pipeline).

* **Alerting**:

  * CloudWatch alarms defined primarily on SLO-related metrics:

    * p95/p99 latency breaches.

    * Error rate thresholds.

    * Unusual drops/spikes in traffic or processing.

  * Notifications to ops channels (email/Slack/incident tool).

(Third-party APM tools like Datadog are reserved for a later growth phase and are not required for initial production.)

A.2.6 Mapping, Routing, and Location Services

* **Primary Mapping & Location Platform**:

  * **AWS Location Service** – for:

    * Geocoding and reverse geocoding.

  * Route calculations.

  * Storing and querying geo-coordinates, geofences, and trackers.

* Benefits:

  * Tight integration with AWS security/IAM.

  * Consolidated billing with other AWS services.

  * Cost-effective, pay-per-use model with free tiers.

* **Driver Navigation**:

  * Mobile apps display maps via AWS Location-backed tiles or equivalent.

  * **Deep-link integration** to **Google Maps or Waze** on the driver's device for turn-by-turn navigation (no per-use navigation API fees).

* **Future/Fallback Providers**:

  * **Google Maps** – used selectively if superior geocoding is required in certain areas.

  * **Mapbox / HERE** – considered for specific future needs, but not core to the initial rollout.

A.2.7 Communication Stack (SMS, Email, Push, WhatsApp)

* **SMS (Transactional)**:

  * A **local South African SMS aggregator** (e.g. SMS South Africa) is used as the **primary SMS provider** for:

  * OTPs and security codes.

  * Critical parcel events (driver arriving, parcel ready for collection).

  * Optimised for **ZAR billing and bulk discounts**, protecting margins.

* **Email**:

  * **Amazon Simple Email Service (SES)** is the **primary email provider** for:

    * Transactional emails (parcel confirmation, delivery confirmations, statements, password resets).

    * Operational alerts where needed.

  * AfroGo uses **AWS Route 53** for DNS and **AWS WorkMail** for internal mailboxes (support@, operations@, etc.).

  * Third-party ESPs like SendGrid are optional for later **marketing automation** and are not required for v1.

* **Push Notifications**:

  * **Firebase Cloud Messaging (FCM)** – direct integration from mobile apps for push notifications (Android/iOS).

  * **Amazon SNS** – used from backend services to fan out events to push, SMS, and email as needed.

* **WhatsApp**:

  * **360dialog** as the **primary WhatsApp Business API provider**, enabling:

    * Customer support conversations.

    * Shipping and delivery notifications for opted-in users.

  * Twilio or other providers remain optional for future multi-channel strategies.

A.2.8 Identity, KYC, and Security Providers

* **Authentication & Authorisation**:

  * **Amazon Cognito User Pools & Identity Pools** – for:

    * Customer, merchant, driver, AfroGo Point, and admin authentication.
    * JWT-based authorisation for APIs.

* **KYC / Identity Verification**:

  * **Smile Identity** – primary KYC provider for:

    * Verifying drivers and AfroGo Point owners (ID + selfie + liveness).
    * High-risk merchants where deeper checks are needed.

* **Secrets & Configuration**:

  * **AWS Secrets Manager** – for API keys, DB credentials, and sensitive config.
  * **SSM Parameter Store** – for non-sensitive configuration and feature flags.

A.2.9 Payments, Accounting, and Financial Integrations

* **Payment Gateway (Customer Payments)**:

  * **PayFast** is the **single primary payment gateway**, providing:

    * Card payments (Visa, MasterCard, etc.).
    * Instant EFT.

  * Scan-to-Pay / QR methods.

  * Keeps integration simple while covering the majority of merchant and consumer needs with one robust provider.

* **Payouts (Drivers, AfroGo Points)**:

  * **Bank EFT payouts** via standard bulk payment files or banking APIs.

  * Optional partnership with a payout provider later as scale and requirements evolve.

* **Accounting**:

  * **Sage Business Cloud Accounting (Sage One)** is the **sole accounting system** integrated with AfroGo:

    * Invoices and payments from Aurora are synced into Sage.

    * Ensures strong local compliance and easy collaboration with South African accountants.

A.2.10 E-Commerce & Platform Integrations

AfroGo is designed to be a **first-class logistics option inside merchants' existing tools**:

* **Shopify**:

  * Native **AfroGo shipping/carrier app**:

    * Provides shipping rate calculations at checkout.

    * Automatically creates AfroGo shipments when orders are placed.

* **WooCommerce**:

* Official **AfroGo WooCommerce plugin**:

  * Adds AfroGo as a shipping method.

  * Sends order/shipping data to AfroGo's APIs.

* **Bob Go and other aggregators**:

  * Direct integration so AfroGo appears as a **courier option inside Bob Go** and similar platforms.

  * Merchants already using aggregators can enable AfroGo with minimal friction.

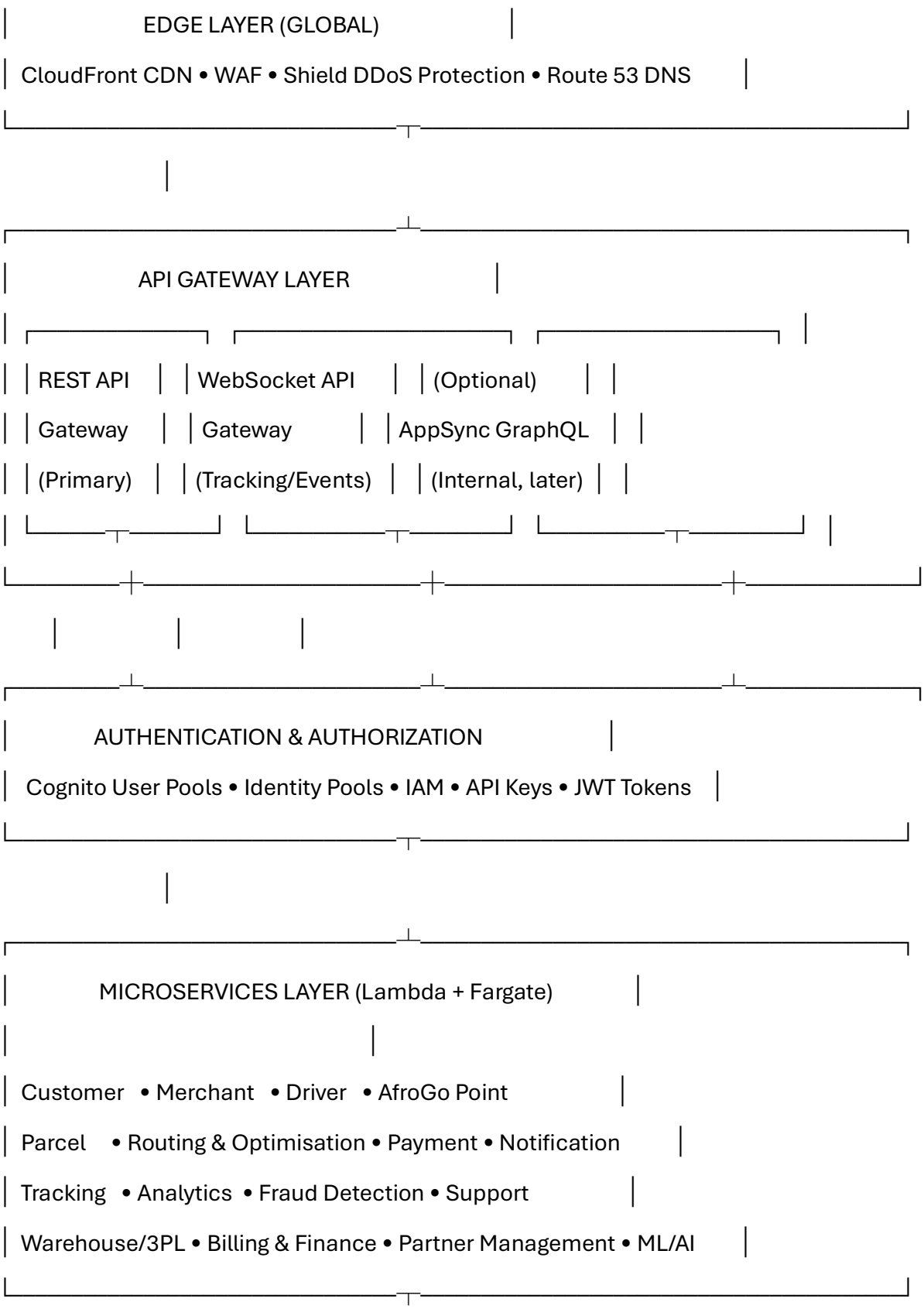
* **Additional Platforms (Magento, custom integrations)**:

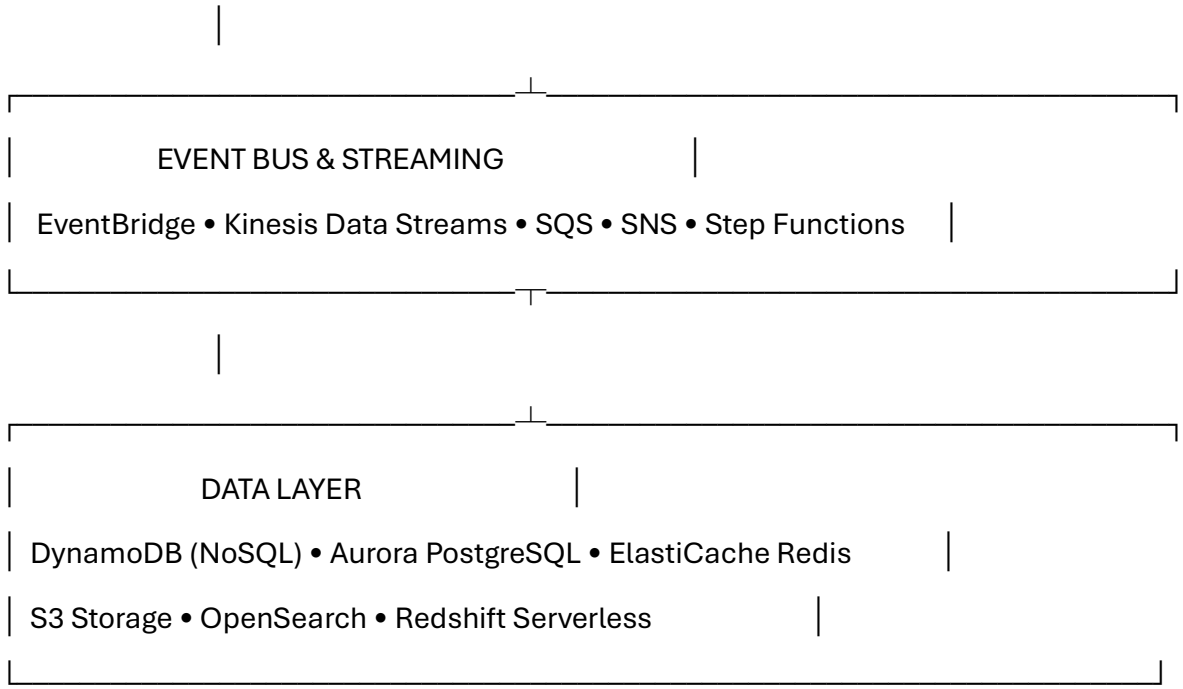
  * Supported via the same E-commerce Integration Service when demand arises.

PART B: DETAILED PLATFORM ARCHITECTURE

B.1 HIGH-LEVEL SYSTEM ARCHITECTURE DIAGRAM

```text
┌
```


EDGE LAYER (GLOBAL)
CloudFront CDN • WAF • Shield DDoS Protection • Route 53 DNS
API GATEWAY LAYER
REST API Gateway (Primary)
WebSocket API Gateway (Tracking/Events)
(Optional) AppSync GraphQL (Internal, later)
AUTHENTICATION & AUTHORIZATION
Cognito User Pools • Identity Pools • IAM • API Keys • JWT Tokens
MICROSERVICES LAYER (Lambda + Fargate)
Customer • Merchant • Driver • AfroGo Point
Parcel • Routing & Optimisation • Payment • Notification
Tracking • Analytics • Fraud Detection • Support
Warehouse/3PL • Billing & Finance • Partner Management • ML/AI




```

* **Primary interaction**: REST APIs via API Gateway + Cognito + Lambda.

* **Realtime flows**: WebSocket API for tracking / ops dashboards; Kinesis + Redis for high-frequency updates.

* **GraphQL/AppSync**: kept **in the toolbox** for future **internal dashboards** or complex querying, not required on day one.

---

## B.2 MICROSERVICES DETAILED BREAKDOWN (WITH FINAL CHOICES)

All business logic is organised into **microservices** (mostly AWS Lambda functions) following **DDD bounded contexts**.

Common tech across services:
```

* Runtime: **Node.js + TypeScript**

* Framework: **NestJS on Fastify**

* Shared library: `@afrogo/core` for:

  * Logging (structured JSON),

  * Auth (Cognito JWT verification),

  * Metrics (CloudWatch),

  * Config.

---

1. Customer Service

Responsibility

Manage customer accounts, profiles, addresses, preferences.

Key APIs

* `POST /customers/register` – Create new customer.

* `POST /customers/login` – Authenticate customer (delegates to Cognito).

* `GET /customers/{id}/profile` – Retrieve profile.

* `PUT /customers/{id}/profile` – Update profile.

* `POST /customers/{id}/addresses` – Add delivery address.

* `GET /customers/{id}/addresses` – List addresses.

* `PUT /customers/{id}/preferences` – Update communication & language prefs.

Data Store

* DynamoDB table: `Customers`

  `{ customerId, email, phone, name, kycStatus, createdAt, addresses[], preferences }`

Events Published

* `CustomerRegistered`
* `CustomerVerified`
* `CustomerProfileUpdated`

Integration Points

* **Cognito** – auth, MFA.
* **Smile Identity** – KYC if/when needed for certain segments.
* **Notification Service** – welcome email/SMS, password reset flows.

---

2. Merchant Service

Responsibility

Business accounts, bulk shipments, API keys, credit terms.

Key APIs

* `POST /merchants/register`
* `POST /merchants/{id}/shipments/bulk`
* `GET /merchants/{id}/shipments`
* `GET /merchants/{id}/analytics`
* `POST /merchants/{id}/api-keys`
* `GET /merchants/{id}/invoices`

Data Store

* DynamoDB: `Merchants`, `MerchantShipments`, `MerchantAPIKeys`
* Aurora PostgreSQL: `MerchantInvoices`

Events Published

* `MerchantOnboarded`
* `ShipmentCreated`
* `MerchantCreditLimitChanged`

Integration Points

* Parcel Service – create parcels.
* Billing & Finance – invoice generation, limits.
* E-commerce integrations – Shopify app, WooCommerce plugin, Bob Go.

---

3. Driver Service

Responsibility

Driver onboarding, vetting, availability, routes, earnings.

Key APIs

* `POST /drivers/register`
* `PUT /drivers/{id}/verify`
* `POST /drivers/{id}/availability`
* `GET /drivers/{id}/routes`
* `POST /drivers/{id}/scans`
* `GET /drivers/{id}/earnings`
* `POST /drivers/{id}/payout-request`

Data Store

* DynamoDB: `Drivers`, `DriverAvailability`, `DriverEarnings`
* S3: Driver docs (ID, license, vehicle papers).

Events Published

* `DriverRegistered`
* `DriverVerified`
* `DriverOnline` / `DriverOffline`
* `ParcelScanned`

* `DeliveryCompleted`

* `DeliveryFailed`

Integration Points

* **Smile Identity** – ID + selfie verification.

* Routing Service – assigning routes.

* Payment/Billing – earnings and payouts (bank EFT).

* Tracking Service – GPS and scan events.

---

4. AfroGo Point Service

Responsibility

Pickup point partners (AfroGo Points): onboarding, parcel handling, commissions.

Key APIs

* `POST /points/register`

* `PUT /points/{id}/verify`

* `GET /points/{id}/parcels`

* `POST /points/{id}/parcels/{parcelId}/receive`

* `POST /points/{id}/parcels/{parcelId}/release`

* `GET /points/{id}/commissions`

Data Store

* DynamoDB: `AfroGoPoints`, `PointParcels`, `PointCommissions`
* Redis: live parcel counts per point.

Events Published

* `PointOnboarded`
* `ParcelArrivedAtPoint`
* `ParcelCollectedByCustomer`
* `PointCommissionAccrued`

Integration Points

* Parcel Service – status updates.
* Payment/Billing – calculating commissions.
* Notification Service – “parcel ready for collection” SMS/WhatsApp/email.

---

5. Parcel Service (Core Domain)

Responsibility

End-to-end parcel lifecycle: creation → delivery/collection → returns.

Key APIs

* `POST /parcels`
* `GET /parcels/{id}`
* `GET /parcels/{id}/status`
* `PUT /parcels/{id}/status` (internal)
* `POST /parcels/{id}/reschedule`
* `POST /parcels/{id}/redirect`
* `GET /parcels/search` (backed by OpenSearch)

Data Store

* DynamoDB: `Parcels`, `ParcelStatusHistory`
* OpenSearch: index for search (by tracking number, name, phone, etc).
* S3: POD photos, signatures.

State Machine

```text
Created → InWarehouse → AssignedToDriver → InTransit → OutForDelivery
  → [Delivered | ArrivedAtPoint | Failed] → [Collected | Returned]
```

Events Published

* `ParcelCreated`
* `ParcelInWarehouse`

* `ParcelAssignedToDriver`
* `ParcelInTransit`
* `ParcelOutForDelivery`
* `ParcelDelivered`
* `ParcelArrivedAtPoint`
* `ParcelCollected`
* `ParcelFailed`
* `ParcelReturned`

SLA Engine

* AfroGo door: **1–3 business days**.
* AfroCollect: **2–4 business days**.
* Hourly Lambda checks SLAs and emits `SLABreachAlert` events.

Integration Points

* Routing Service – parcel batching and route assignment.
* Tracking Service – status → tracking views.
* Notification Service – status change notifications.
* Billing/Finance – revenue recognition and invoicing.

---

6. Routing & Optimization Service

Responsibility

Route planning, clustering, assignment, dynamic re-optimisation.

Core Logic

* Clustering by area (e.g. Soweto loop, Pretoria East loop).

* Vehicle capacity, time window constraints.

* Re-optimisation if:

  * New parcels added.

  * Driver goes offline.

  * Traffic issues detected.

Algorithms

* Use **Google OR-Tools** for VRP/TSP solving.

* Distances/ETAs primarily via **AWS Location Service**; selectively use Google APIs if needed.

* Historical travel times feed **SageMaker models** for travel-time prediction.

Key APIs

* `POST /routing/optimize`

* `GET /routing/routes/{routeId}`

* `POST /routing/routes/{routeId}/reassign`

* `GET /routing/driver/{driverId}/navigation` (high-level route, not full nav – app then deep-links to Google/Waze).

Data Store

* DynamoDB: `Routes`, `RouteStops`
* Redis: `drivers:live` geospatial data.
* S3: historical route snapshots.

ML Models (SageMaker)

* Demand forecasting per area/day.
* Travel-time prediction.
* Driver performance prediction.

Events Published

* `RouteCreated`
* `RouteOptimized`
* `RouteAssignedToDriver`
* `RouteCompleted`

---

7. Payment Service

Responsibility

Customer payments, refunds, and feeding payout data to Billing/Finance.

Key APIs

* `POST /payments/charge` – via **PayFast** (cards + Instant EFT + Scan-to-Pay).
* `GET /payments/{transactionId}` – payment status.
* `POST /payments/refund` – trigger refund via PayFast.
* `GET /payments/reconciliation` – reports for finance.

Payment Methods (via PayFast)

* Credit/debit cards.
* Instant EFT.
* Certain QR/scan-to-pay methods, depending on PayFast configuration.

Data Store

* Aurora PostgreSQL: `Transactions`, `Refunds`.
* DynamoDB: event log for idempotency and quick checks.

Payouts (Drivers & AfroGo Points)

* Calculated by **Billing & Finance Service**.
* Executed as **bulk bank EFTs** (via your bank or a payout API partner if adopted later).

Events Published

* `PaymentReceived`

* `PaymentFailed`

* `RefundProcessed`

Fraud Integration

* Sends transaction metadata to Fraud Service for scoring.

---

8. Notification Service

Responsibility

Multi-channel comms: SMS, email, push, WhatsApp.

Channels

* **SMS**: Local SA SMS aggregator (e.g. SMS South Africa) – cost-effective in ZAR.

* **Email**: **AWS SES** – transactional mail only (no SendGrid initially).

* **Push**: **Firebase Cloud Messaging (FCM)** from the apps; backend uses **SNS** to fan out events.

* **WhatsApp**: **360dialog** WhatsApp Business API (main WA channel).

Key APIs

* `POST /notifications/send` – abstracted “send notification” call (service chooses channels based on user preferences).

* `GET /notifications/{id}/status`
* `POST /notifications/templates`

Typical Triggers

* `ParcelCreated` → confirmation email + optional SMS.
* `ParcelOutForDelivery` → SMS/WhatsApp/push.
* `ParcelArrivedAtPoint` → SMS/WhatsApp with OTP + pickup details.
* `ParcelDelivered` → email/push + feedback link.
* `SLABreachAlert` → internal ops channel.

Data Store

* DynamoDB: `NotificationLog`, `Templates`, `UserPreferences`.
* SQS: work queue for notifications (retries, rate limiting).

Smart Behaviour

* Quiet hours for non-critical SMS.
* Per-user channel preferences.
* Multi-language templates (matching UI languages).

---

9. Tracking Service

Responsibility

Live parcel & driver location + ETAs.

Key APIs

* `GET /tracking/{trackingNumber}` – public tracking data.
* `GET /tracking/parcels/{parcelId}/location`
* `GET /tracking/drivers/{driverId}/location`
* `POST /tracking/drivers/{driverId}/location` – driver app telemetry.
* `GET /tracking/parcels/{parcelId}/eta`

Tech

* WebSocket or IoT Core for driver → backend location updates (every ±30 seconds).
* Redis geospatial for fast “driver near X?” queries.
* DynamoDB for location history.
* AWS Location Service for distance/route ETAs, with ML refinement.

Customer Tracking Page

`https://track.afrogo.africa/{trackingNumber}` (example):

* Status timeline.
* Map with current parcel/driver location (if en route).
* Estimated delivery window.
* AfroGo Point details if applicable.

Events Published

* `DriverLocationUpdated`
* `ParcelLocationUpdated`
* `ETAUpdated`

---

10. Analytics Service

Responsibility

BI, performance metrics, and dashboards.

Metrics

* Volumes by day/week/month, by service type (AfroGo vs AfroCollect).
* Revenue, margins, cost lines (driver, points, 3PL, communications).
* OTD%, failed delivery breakdowns, reasons.
* Driver & point performance.
* Merchant and customer retention.

Pipeline

```text
Operational DBs (DynamoDB, Aurora)
```

→ Kinesis Firehose

→ S3 Data Lake (raw)

→ AWS Glue ETL

→ Redshift Serverless (warehouse)

→ QuickSight Dashboards

```

**Dashboards**

* **Executive** – revenue, parcels, margins, SLA.
* **Ops** – live operational view (combined with Tracking).
* **Merchants** – their volumes, on-time %, billing.
* **Drivers/Points** – performance & earnings dashboards.

**Data Stores**

* S3 (raw & processed).
* Redshift Serverless (star schema).
* OpenSearch for operational searches.

---

### 11. Fraud Detection Service

**Responsibility**

Fraud prevention across payments, parcels, drivers, and merchants.
```

Vectors

* Payment fraud, fake merchants, driver/point collusion, account takeovers.

Mechanisms

* Rule engine:

  * Velocity limits, new customer/merchant anomalies.

  * Mismatched address patterns.

* ML (SageMaker):

  * Anomaly detection.

  * Graph-based detection of suspicious networks.

  * Computer vision on delivery photos.

Key APIs

* `POST /fraud/check-transaction`

* `POST /fraud/report`

* `GET /fraud/cases`

Actions

* Dynamic scoring:

  * 0–30 → auto approve.

  * 31–70 → manual review.

  * 71–100 → hard block + suspension.

Data

* DynamoDB: `FraudCases`, `FraudScores`.

* S3: evidence.

* OpenSearch: searchable fraud case history.

---

12. Support Service (Customer Service & Helpdesk)

Responsibility

Tickets, chat, escalations, knowledge base.

Channels

* In-app/web chat (initially can be simple widget or custom minimal chat; later AWS Connect or a CRM).

* Email: **support@afrogo…** (WorkMail + SES).

* WhatsApp: via **360dialog** (customer support line).

* FAQs and help centre on the website.

Features

* Ticket lifecycle: open → in-progress → resolved.
* Auto-link tickets to parcels/merchants/drivers.
* AI assistant (via Bedrock) to suggest replies and auto-answer simple queries.
* Sentiment analysis (Comprehend or custom).

APIs

* `POST /support/tickets`
* `GET /support/tickets/{id}`
* `POST /support/tickets/{id}/messages`
* `PUT /support/tickets/{id}/resolve`
* `GET /support/kb/search`

Data

* DynamoDB: `Tickets`, `TicketMessages`.
* OpenSearch: tickets + knowledge base.
* S3: attachments.

---

13. Warehouse (3PL) Integration Service

Responsibility

Integration with Parcelninja (initially) and future 3PLs.

Key Flows

* Inbound stock notifications.
* Outbound pick/pack orders.
* Returns handling.
* Inventory snapshots (for SMEs using fulfilment).

APIs (AfroGo side)

* `POST /3pl/inbound`
* `GET /3pl/inventory/{sku}`
* `POST /3pl/outbound`
* `GET /3pl/outbound/{orderId}/status`
* `POST /3pl/returns`

Data

* DynamoDB: `3PLOrders`, `InventorySnapshot`.
* S3: documents from 3PL, reports.

---

14. Billing & Finance Service

Responsibility

Invoicing, payouts, revenue recognition, accounting sync.

Workflows

* Merchant invoicing & statements.
* Driver weekly payout calculations.
* AfroGo Point commission calculations.
* Revenue recognition (parcel delivered/collected).

APIs

* `GET /billing/merchants/{id}/invoices`
* `POST /billing/invoices/{id}/payment`
* `GET /billing/drivers/{id}/payout`
* `POST /billing/process-payouts`
* `GET /billing/reports/revenue`

Data

* Aurora PostgreSQL: `Invoices`, `InvoiceLineItems`, `Payments`, `Payouts`.
* S3: invoice PDFs, payout files.

Accounting

* Syncs with **Sage Business Cloud Accounting (Sage One)** only:

* Invoices & payments pushed to Sage via its API.

* Sage is the source of truth for GL and tax.

---

15. Partner Management Service

Responsibility

Lifecycle of drivers and AfroGo Points as partners.

Features

* Application workflows.

* Document upload + Smile Identity KYC.

* Online training content + quizzes.

* Digital contracts (e.g. via DocuSign).

* Performance scoring + incentives.

APIs

* `POST /partners/drivers/apply`

* `PUT /partners/drivers/{id}/verify`

* `GET /partners/drivers/{id}/performance`

* `POST /partners/points/apply`

* `GET /partners/points/{id}/performance`

Data

* DynamoDB: `PartnerApplications`, `PartnerPerformance`.
* S3: documents.
* Aurora: `PartnerContracts`.

---

16. ML/AI Service

Responsibility

Central ML/AI pipeline and inference endpoints.

Models

* Demand forecasting (by area/day).
* Delivery time prediction.
* Churn prediction (merchants/customers).
* Fraud models (payment, behaviour).
* Sentiment analysis (support).
* Image recognition for delivery photos.

GenAI via Bedrock

* AI support/copilot for ops and support.

* Auto-generated merchant insights and recommendations.

APIs

* `POST /ml/predict/demand`
* `POST /ml/predict/delivery-time`
* `POST /ml/predict/churn`
* `POST /ml/analyze/sentiment`
* `POST /ml/recognize/image`

Data

* S3: training datasets.
* SageMaker Feature Store.
* Model Registry + SageMaker endpoints.

B.3 EVENT-DRIVEN ARCHITECTURE

Same concept as your original, but locked to our stack:

* **Event bus**: Amazon EventBridge (business events).
* **Streams**: Kinesis (GPS, scan events, high-volume telemetry).
* **Consumers**: Lambdas for reactions, Firehose to S3, analytics jobs.

Parcel creation, delivery completion, and SLA breach flows remain as you wrote, just with the updated services.

B.4–B.9 (Data, Security, IaC, CI/CD, Monitoring, DR)

These sections are **structurally the same as your original**, with these strategic changes already reflected:

* **Data layer**: DynamoDB + Aurora + Redis + S3 + OpenSearch + Redshift – unchanged.
* **Security**: Cognito + IAM + KMS + WAF + Shield + POPIA + no card storage – unchanged.
* **IaC**: AWS CDK (TypeScript) for everything; Terraform only if multi-cloud later.
* **CI/CD**: GitHub Actions as primary; CodePipeline optional.
* **Monitoring**:
  * Use **CloudWatch + X-Ray + OpenSearch** as main observability stack.
  * Datadog/APM is **not used at launch**, can be added later if needed.
* **DR/Backups**: PITR, snapshots, cross-region copies – unchanged.